American University of Sharjah
College of Engineering

TinyML Project – Milestone 3 Report

On-Device Prototype and Optimization

Energy-Efficient Human Activity Recognition on the Edge

Team Member	AUS ID
Lee Ting Sen	B00103724
Khalifa Alshamsi	B00078654
Vineetha Addanki	G00111196

Project Track	Track B: UCI-HAR public benchmark with Arduino target-domain adaptation and live on-device evidence.
Board / Sensor	Arduino Nano 33 BLE Sense with onboard LSM9DS1 accelerometer and gyroscope.
Selected M3 Build	`m3_v2_1_mixed_6ch`: mixed UCI + Arduino DS-CNN, full-integer INT8, deployed through `model_data.h`.
Main Live Result	Right-pocket controlled live Serial: 0.9040 accuracy / 0.9089 macro F1. Left-pocket robustness live Serial: 0.8901 accuracy / 0.8826 macro F1.

May 2026

Executive Summary

Milestone 3 answers whether the human activity recognition model runs on an Arduino with live sensor input. The final evidence-collection build is `m3_v2_1_mixed_6ch`, a 6-channel depthwise-separable CNN trained from UCI-HAR source data plus Arduino V2/V2.1 target-domain adaptation data, then converted to a full-integer INT8 TFLite Micro C array. The board reads live LSM9DS1 accelerometer and gyroscope samples, forms 128-sample windows with 64-sample stride at 50 Hz, normalizes and quantizes inputs on-device, invokes TFLite Micro, and prints parseable Serial predictions.

The deployment evidence satisfies the main M3 requirements: the sketch compiles and runs on Arduino Nano 33 BLE Sense, model size is 10,288 bytes, flash usage is 177,504 bytes, RAM usage is 111,144 bytes, tensor arena size is 61,440 bytes, and average `Invoke()` latency is 34.113 ms over 50 calls. True live Serial testing achieved 0.9040 accuracy / 0.9089 macro F1 under the controlled right-pocket condition and 0.8901 accuracy / 0.8826 macro F1 under the left-pocket robustness condition. The main remaining failure is stair-direction confusion, especially `WALKING_UPSTAIRS -> WALKING_DOWNSTAIRS`. The model is therefore a working M3 prototype and evidence-collection build, not a fully robust final HAR system.

1 R1 Deployment Pipeline

1.1 System Path

The project remains Track B because UCI-HAR is the public source-domain benchmark and official held-out test reference. Arduino V2 and V2.1 are used to adapt to the deployment distribution. Final live evidence comes from Arduino Serial predictions and is kept separate from laptop replay validation.

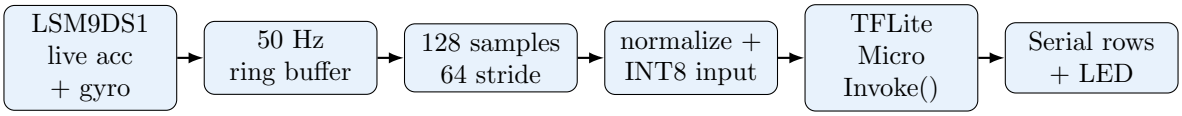

Figure 1: Live Arduino inference pipeline. Training uses TensorFlow/Keras, conversion uses full-integer INT8 TFLite, and deployment uses a TFLite Micro C array.

1.2 Model Conversion and Arduino Sketch

The conversion path is:

`TensorFlow/Keras -> full-integer INT8 TFLite PTQ -> C array in model_data.h -> TFLite Micro on Arduino`

The selected model is packaged into `arduino/tinyml_har_m3/model_data.h`. The header stores the INT8 model bytes, input/output quantization parameters, class labels, and six-channel normalization constants. The Arduino sketch `arduino/tinyml_har_m3/tinyml_har_m3.ino`:

- reads accelerometer x/y/z and gyroscope x/y/z from LSM9DS1,
- targets 50 Hz sampling using scheduled timing,
- converts gyroscope values from degrees/second to radians/second,
- stores a 128-sample ring buffer and runs inference every 64 new samples,
- applies the exported training/adaptation normalization constants,
- quantizes normalized inputs using the TFLite input scale and zero point,
- calls `interpreter->Invoke()` and measures latency with `micros()`,
- prints timestamp, window id, prediction id, label, confidence, latency, top-1 score, top-2 label/score, and running average latency.

Table 1: Arduino deployment environment.

Item	Version / description
Board	Arduino Nano 33 BLE Sense.
Sensor	Onboard LSM9DS1 accelerometer and gyroscope.
Arduino IDE	2.3.8.
Board package	Arduino Mbed OS Nano Boards 4.5.0.
Sensor library	Arduino_LSM9DS1 1.1.1.
TinyML library	Harvard_TinyMLx 1.2.4-Alpha.
Primary output	Serial monitor prediction rows; simple LED state as auxiliary feedback.

2 R2 Deployment Metrics

Deployment metrics were measured from the actual Arduino compile/run evidence. Laptop replay and laptop latency are not counted as Arduino hardware metrics.

Table 2: Measured deployment metrics for the selected INT8 Arduino candidate.

Metric	Value	Evidence / method
INT8 `.tflite` model size	10,288 bytes / 10.05 KB	Selected deployment file `m3_v2_1_mixed_6ch_int8.tflite`.
`model_data.h` file size	65,932 bytes	Generated C array and constants.
Tensor arena size	61,440 bytes / 60.00 KB	Boot Serial metadata.
Flash usage	177,504 bytes / 173.34 KB / 18%	Arduino IDE compile output.
Global dynamic memory	111,144 bytes / 108.56 KB / 42%	Arduino IDE compile output.
Average `Invoke()` latency	34,113 us / 34.113 ms	Serial latency summary averaged over 50 inferences.
Stability	Passed, about 156 seconds	Returned stability video evidence.
Demo evidence	At least three proof clips plus Serial logs	Sitting, standing, walking proof clips and live trial logs.

The model fits comfortably within the Arduino Nano 33 BLE Sense memory limits. Inference is triggered once per completed stride window rather than at every raw sensor sample, so the 34.113 ms `Invoke()` time is acceptable for this windowed HAR prototype.

3 R3 Optimization: Baseline vs. Improved

3.1 Why the Optimization Changed Direction

The M2 lightweight DS-CNN was strong on the UCI-HAR public benchmark, but it did not transfer to the real Arduino environment. The UCI-trained baseline predicted all Arduino V2 replay windows as `LAYING`, giving only 0.1864 accuracy and 0.0524 macro F1 on 236 windows. This showed that the main M3 problem was not just model size: it was UCI-to-Arduino domain shift caused by hardware, orientation, pocket placement, clothing, and normalization mismatch.

Therefore, R3 is organized as one optimization trail rather than three repeated experiment sections:

1. **M2 baseline:** UCI-only DS-CNN, strong public benchmark, poor Arduino transfer.
2. **M3 target-domain search:** compare UCI-only, Arduino-only, pretrain/fine-tune, mixed, focal, and gravity candidates.
3. **M3 final packaging:** select a UCI-retaining mixed 6-channel candidate, quantize it to INT8, deploy it, and verify live Arduino behavior.

3.2 Before/After Result

Table 3: Required before/after comparison between M2 baseline and M3 improved candidate.

Metric	M2 UCI-only DS-CNN	M3 V2.1 mixed 6-channel INT8	Change / tradeoff
UCI-HAR test accuracy	0.9169	0.7984	Lower public-source score after adapting to Arduino data.
UCI-HAR test macro F1	0.9173	0.7960	Lower public-source score, accepted to improve target behavior.
Arduino V2 grouped macro F1	UCI-only replay collapsed	0.7309	Large target-domain improvement.
Held-out right_60s replay macro F1	Not measured for M2	0.7741	Strong controlled replay candidate.
Held-out left_30s replay macro F1	Not measured for M2	0.3886	Raw replay robustness gap.
True right-pocket live macro F1	Not measured for M2	0.9089	Live Arduino controlled evidence.
True left-pocket live macro F1	Not measured for M2	0.8826	Live Arduino robustness evidence.
Deployment TFLite size	13,460 bytes	10,288 bytes	INT8 reduces deployment model size.
Arduino latency / RAM	Not measured for M2	34.113 ms / 61,440-byte arena	Real deployment metrics added.

The optimization is meaningful because it addresses the actual deployment failure. UCI-HAR performance drops, but Arduino-domain validation and live evidence improve substantially. The report does not claim that the M3 model is universally better; it is better for the target Arduino evidence-collection setting.

3.3 Six-Model Target-Domain Comparison

V2.1 added four 5-minute Arduino adaptation captures for `WALKING`, `SITTING`, `STANDING`, and `LAYING`. These were used only for training/adaptation, with capped sampling so non-stair classes would not overwhelm the stair classes. Held-out `right_60s` and `left_30s` raw sensor validation were not used for training, normalization fitting, representative quantization, or early stopping.

Table 4: Final V2.1 six-model comparison. Values shown are INT8 macro F1 where an INT8 candidate was evaluated.

Model family	UCI F1	V2 Group F1	Right 60s F1	Left 30s F1	Decision
UCI-only baseline	0.9025	0.0587	0.0526	0.0535	Reject: public benchmark only; Arduino replay collapses.
Arduino-only	0.1082	0.4686	0.8050	0.5094	Reject: high right replay but destroys UCI retention and still has posture failures.
UCI pretrain + V2.1 fine-tune	0.4630	0.1986	0.2582	0.3442	Reject: expected transfer path, but weak target validation.
Mixed 6-channel DS-CNN	0.7960	0.7309	0.7741	0.3886	Selected M3 controlled-demo build.
Mixed focal 6-channel DS-CNN	0.7396	0.7099	0.7357	0.5643	Intermediate: better left replay than selected model, but weaker controlled right replay.
Mixed 10-channel gravity DS-CNN	0.8832	0.6189	0.5655	0.6856	Robustness candidate; not selected because right replay is lower and 10-channel Arduino deployment is more complex.

The selected model is `m3_v2_1_mixed_6ch`. It was selected because the M3 controlled demo priority is right-pocket behavior, and this model was the strongest UCI-retaining mixed candidate for the controlled right-pocket replay. The left-pocket raw replay weakness is kept as a robustness warning, not hidden. The 10-channel gravity model is a useful M4 direction, but the simpler 6-channel path was chosen because it matches the deployed Arduino feature pipeline and achieved better controlled-demo validation.

4 R4 On-Device Accuracy

The handout requires live Arduino predictions for on-device accuracy. The table below separates public laptop benchmark, raw sensor replay validation, and true live Arduino Serial results.

Table 5: Offline benchmark, raw replay validation, and true live Arduino evaluation.

Evaluation setting	Acc.	Macro F1	Weighted F1	Evidence type
UCI-HAR official test, M2 baseline	0.9169	0.9173	0.9168	Laptop public benchmark.
UCI-HAR official test, M3 INT8 candidate	0.7984	0.7960	–	Laptop TFLite benchmark.
Arduino V2 grouped replay, M3 INT8	0.7573	0.7309	–	Held-out grouped raw sensor replay.
right_60s replay, M3 INT8	0.8708	0.7741	–	Held-out controlled raw sensor replay.
Right-pocket controlled live Serial	0.9040	0.9089	0.8999	Arduino live inference, 125 scored rows.

The right-pocket controlled live trial used at least 20 rows per class. Static classes were perfect in the returned logs. The only major error was `WALKING_UPSTAIRS -> WALKING_DOWNSTAIRS`.

Table 6: Right-pocket controlled live confusion matrix and per-class F1.

Actual \Predicted	Walk	Up	Down	Sit	Stand	Lay	F1
WALKING	20	0	0	0	0	0	1.0000
WALKING_UPSTAIRS	0	13	12	0	0	0	0.6842
WALKING_DOWNSTAIRS	0	0	20	0	0	0	0.7692
SITTING	0	0	0	20	0	0	1.0000
STANDING	0	0	0	0	20	0	1.0000
LAYING	0	0	0	0	0	20	1.0000

The controlled live score is higher than the public UCI-HAR score of the adapted model because the live trials were performed under the deployment protocol and fixed placement. It is not inflated from replay: it was computed from Arduino Serial prediction logs generated during live sensor inference.

5 R5 Robustness Test

The robustness condition is left-pocket placement, a new orientation and placement condition relative to the controlled right-pocket setup. It tests whether the model remains stable when the board is worn differently.

Table 7: Controlled and robustness comparison.

Condition	Acc.	Macro F1	Evidence type	Main failure mode
right_60s replay, INT8	0.8708	0.7741	Raw sensor replay	`WALKING_UPSTAIRS -> WALKING_DOWNSTAIRS`.
left_30s replay, INT8	0.4348	0.3886	Raw sensor replay	`SITTING -> LAYING`; `LAYING -> STANDING`.
Right-pocket live Serial	0.9040	0.9089	Arduino live inference, 125 rows	`WALKING_UPSTAIRS -> WALKING_DOWNSTAIRS`.
Left-pocket live Serial	0.8901	0.8826	Arduino live inference, 91 rows	`WALKING_UPSTAIRS -> WALKING_DOWNSTAIRS`.

Table 8: Left-pocket robustness live confusion matrix and per-class F1.

Actual \Predicted	Walk	Up	Down	Sit	Stand	Lay	F1
WALKING	10	0	0	0	0	0	0.9091
WALKING_UPSTAIRS	0	6	6	0	0	0	0.6667
WALKING_DOWNSTAIRS	2	0	18	0	2	0	0.7826
SITTING	0	0	0	22	0	0	1.0000
STANDING	0	0	0	0	15	0	0.9375
LAYING	0	0	0	0	0	10	1.0000

The live robustness result is much stronger than earlier left-pocket raw replay. This shows why the final M3 evidence must use live Arduino Serial predictions. The dominant live robustness issue remains stair-direction separation, not static posture separation.

5.1 Deployment Placement Guidance

Teammate testing showed that the model is sensitive to board placement and orientation. The recommended controlled setup is the right pocket. If left pocket is used, the board should be oriented to resemble right-pocket placement as closely as possible. Static postures depend strongly on gravity/board angle: standing worked best with the board facing downward at about 90 degrees, sitting with a diagonal angle around 135 degrees, and laying flatter/horizontal around 180 degrees. Walking should be straight and natural; excessive leg lifting can push predictions toward `WALKING_DOWNSTAIRS`. These are deployment findings, not hidden bugs, and they explain the remaining confusion matrices.

6 R6 Challenges and Lessons Learned

The largest challenge was the public UCI-HAR to Arduino domain shift. A model that performed well on UCI-HAR collapsed to `LAYING` on Arduino V2 replay. This happened for both FP32 and INT8, so quantization was not the root cause. The likely causes were sensor hardware differences, board orientation, pocket placement, clothing movement, and normalization mismatch. A domain audit found Arduino V2 `acc_x` about -3.40 standard deviations from the UCI-HAR training standardizer mean.

The second challenge was evidence separation. The project had to keep four different evidence types distinct: UCI-HAR public benchmark, Arduino V2 grouped replay, held-out right/left raw replay, and true live Arduino Serial predictions. Only the last one is on-device accuracy. This separation prevents the report from overclaiming laptop replay as live performance.

The third challenge was robustness. Model size and latency were acceptable after INT8 conversion, but board placement remained sensitive. Static postures can be classified largely from gravity, which helps under fixed placement but makes predictions sensitive to angle changes. Dynamic walking classes are sensitive to gait style, and stair direction is still difficult.

If starting again, the team would define fixed board orientation earlier, collect calibration and stair data before model selection, and keep right/left placement protocols consistent from the beginning.

7 R7 Plan for M4

For M4, the goal is to turn the M3 evidence-collection build into a more robust demonstration model.

Table 9: M4 improvement plan.

#	Planned action	Reason
1	Collect more balanced live upstairs and downstairs examples across both pockets.	Main live failure is stair-direction confusion.
2	Standardize board orientation before each run.	Placement sensitivity is the main practical limitation.
3	Add a short calibration pose or orientation-normalization step.	Helps align gravity direction across pockets and users.
4	Explore orientation-invariant features or gravity-frame alignment.	May reduce dependence on board angle.
5	Revisit the 10-channel gravity candidate.	It showed better left-pocket raw replay but requires a more complex Arduino path.
6	Test with more users, days, and environments.	Current evidence is strong for M3 but still limited.
7	Use QAT only if future PTQ becomes unstable.	Current INT8 PTQ does not materially degrade the selected model.

The final repository contains the Arduino sketch, `model_data.h`, selected INT8 model, training and packaging scripts, live scoring script, README instructions, report draft, hardware metrics, live Serial confusion matrices, and the M3 submission checklist. The remaining submission steps are to export this report to PDF and attach the demo video or Serial evidence required by the handout.